

# Recording LFP with NetPyNE and CoreNEURON

September 9, 2026

## Contents

|           |                                                                             |           |
|-----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Quick start</b>                                                          | <b>2</b>  |
| 1.1       | The problem . . . . .                                                       | 2         |
| 1.2       | The solution in one paragraph . . . . .                                     | 2         |
| <b>2</b>  | <b>Files included in the workaround</b>                                     | <b>2</b>  |
| <b>3</b>  | <b>Installing the customized files</b>                                      | <b>3</b>  |
| <b>4</b>  | <b>CoreNEURON, SONATA, and parallel HDF5 requirements</b>                   | <b>3</b>  |
| 4.1       | Tested software stack . . . . .                                             | 3         |
| 4.2       | Install parallel HDF5 first . . . . .                                       | 4         |
| 4.3       | Build libsonatareport against the parallel HDF5 . . . . .                   | 4         |
| 4.4       | Build NEURON/CoreNEURON with reporting enabled . . . . .                    | 5         |
| 4.5       | Runtime checks . . . . .                                                    | 5         |
| <b>5</b>  | <b>Running and locating the LFP output</b>                                  | <b>6</b>  |
| 5.1       | Minimal simulation configuration . . . . .                                  | 6         |
| 5.2       | Quick MPI validation . . . . .                                              | 6         |
| 5.3       | Importing the LFP into the NetPyNE output . . . . .                         | 7         |
| 5.4       | Reading the CoreNEURON SONATA report . . . . .                              | 8         |
| 5.5       | Putting the LFP into the normal NetPyNE output . . . . .                    | 9         |
| <b>6</b>  | <b>How the workaround works internally</b>                                  | <b>10</b> |
| 6.1       | Traditional NetPyNE LFP path . . . . .                                      | 10        |
| 6.2       | New CoreNEURON path . . . . .                                               | 10        |
| <b>7</b>  | <b>Details of the setup.py modification</b>                                 | <b>10</b> |
| <b>8</b>  | <b>Details of coreneuron_lfp.py</b>                                         | <b>11</b> |
| 8.1       | Transfer-resistance conversion . . . . .                                    | 11        |
| 8.2       | Section classes . . . . .                                                   | 11        |
| 8.3       | Report configuration . . . . .                                              | 11        |
| <b>9</b>  | <b>Details of the run.py modification</b>                                   | <b>12</b> |
| <b>10</b> | <b>Voltage-trace recording with CoreNEURON</b>                              | <b>12</b> |
| <b>11</b> | <b>Current limitations</b>                                                  | <b>12</b> |
| 11.1      | GPU-native LFP . . . . .                                                    | 12        |
| 11.2      | Features intentionally not supported by this first implementation . . . . . | 13        |

# 1 Quick start

## 1.1 The problem

NetPyNE’s traditional LFP workflow was designed around the standard NEURON execution path. In that workflow, NetPyNE obtains the transmembrane current of each compartment during the simulation and evaluates

$$V_{\text{LFP}}(t) = R I_{\text{mem}}(t), \quad (1)$$

where  $R$  is the extracellular transfer-resistance matrix and  $I_{\text{mem}}$  is the vector of compartment membrane currents. When the model is transferred to CoreNEURON, NetPyNE’s usual Python-side, per-recording-step LFP calculation is no longer the appropriate runtime path. The workaround in this repository therefore moves the runtime LFP calculation into CoreNEURON itself.

## 1.2 The solution in one paragraph

The customized NetPyNE files first let NetPyNE calculate the cell geometry, electrode geometry, and transfer-resistance matrix exactly as it normally would. Those transfer factors are then reorganized into CoreNEURON’s per-segment mapping format and registered through the CoreNEURON mapping interface. A small helper writes the CoreNEURON report configuration, and the modified run logic passes that configuration to CoreNEURON before the simulation begins. CoreNEURON then writes the LFP as a native SONATA/HDF5 report.

For MPI simulations, the SONATA reporter must be linked against a *parallel* HDF5 build. A serial HDF5 installation makes `libsonatareport` fall back to its serial reporting implementation and can leave the final report with contributions from only a subset of MPI ranks. The installation checks for parallel HDF5 are therefore part of this tutorial.

When LFP is required, the currently validated configuration is:

```
cfg.coreneuron = True
cfg.gpu = False
```

GPU execution can still be used for runs where LFP is not required.

# 2 Files included in the workaround

The workaround uses three customized files inside NetPyNE’s `sim` package:

| File                           | Purpose                                                                                                                                                                                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>setup.py</code>          | Extends LFP setup. NetPyNE calculates the transfer-resistance matrices for all local compartment cells and, in CoreNEURON mode, registers native CoreNEURON LFP mappings instead of building the traditional NetPyNE pointer-based LFP recorder.                      |
| <code>coreneuron_lfp.py</code> | New helper module. Converts NetPyNE’s electrode-by-segment transfer matrices into per-segment CoreNEURON factors, registers them, writes <code>report.conf</code> and <code>sim.conf</code> , and stores the generated report paths in the NetPyNE simulation object. |
| <code>run.py</code>            | Passes the generated CoreNEURON simulation/report configuration to <code>neuron.coreneuron</code> before execution. The traditional NetPyNE LFP callback remains disabled in CoreNEURON mode.                                                                         |

The intended repository folder is therefore similar to:

```
LFP_CoreNeuron_fix/
|-- setup.py
|-- run.py
|-- coreneuron_lfp.py
|-- NetPyNE_CoreNEURON_LFP_Fix_Tutorial.pdf
```

### 3 Installing the customized files

Activate the environment used for the simulation and locate the installed NetPyNE directory:

```
conda activate core_neuron

python - <<'PY'
import os
import netpyne
print(os.path.dirname(netpyne.__file__))
PY
```

The returned path will normally end in something similar to

```
.../site-packages/netpyne
```

The customized files belong in:

```
.../site-packages/netpyne/sim/
```

Before replacing anything, back up the original files.

From the repository root:

```
cp LFP_CoreNeuron_fix/setup.py \
  "$NETPYNE_DIR/sim/setup.py"

cp LFP_CoreNeuron_fix/run.py \
  "$NETPYNE_DIR/sim/run.py"

cp LFP_CoreNeuron_fix/coreneuron_lfp.py \
  "$NETPYNE_DIR/sim/coreneuron_lfp.py"
```

The helper must be placed inside `netpyne/sim/` because the customized `setup.py` imports it as a package-local module.

## 4 CoreNEURON, SONATA, and parallel HDF5 requirements

The LFP helper in this repository uses CoreNEURON’s native SONATA reporting path. For a single-process run, ordinary HDF5 is sufficient, but for an MPI simulation the reporting library must be compiled with MPI support *and* against a parallel HDF5 library.

**Important.** Setting `SONATA_REPORT_ENABLE_MPI=ON` is not enough if the HDF5 installation is serial. In that situation `libsonatareport` falls back to its serial reporting implementation. In our validation this produced a correctly structured HDF5 file but only the cells owned by one MPI rank were present, which changed the amplitude and baseline of the final network LFP.

### 4.1 Tested software stack

The following versions were used for the validated CPU/MPI workflow:

| Software          | Tested version/configuration |
|-------------------|------------------------------|
| Python            | 3.10                         |
| NetPyNE           | 1.1.1                        |
| NEURON/CoreNEURON | 9.0.1                        |
| Open MPI          | 5.0.10                       |
| HDF5              | 2.2.0, mpi_openmpi build     |
| libsonatareport   | 2.0.0, MPI reporting enabled |

Other compatible versions may work, but the important requirements are the same: MPI-enabled CoreNEURON, parallel HDF5, and a `libsonatareport` build that detects that parallel HDF5 library.

## 4.2 Install parallel HDF5 first

If Conda is used, the simplest approach is to install the Open MPI variant of HDF5 before compiling `libsonatareport`. In the tested environment:

```
conda activate core_neuron

conda install -c conda-forge \
    "hdf5=2.2.0=mpi_openmpi_*" \
    "openmpi=5.0.10"
```

Verify that the HDF5 installation is parallel-enabled:

```
which h5pcc
h5pcc -showconfig | grep -E 'HDF5 Version|Parallel HDF5'
```

The output must contain:

Parallel HDF5: ON

If it reports `OFF`, do not continue with the SONATA build yet.

The Python `h5py` package does not need MPI support for this workflow. CoreNEURON and `libsonatareport` perform the parallel write during the simulation; Python uses `h5py` only afterward to read the completed report.

## 4.3 Build libsonatareport against the parallel HDF5

If `libsonatareport` was previously compiled while serial HDF5 was installed, use a fresh build directory so that CMake does not reuse the old HDF5 detection.

```
cd /path/to/libsonatareport
rm -rf build

cmake -S . -B build \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX="$CONDA_PREFIX" \
    -DSONATA_REPORT_ENABLE_SUBMODULES=ON \
    -DSONATA_REPORT_ENABLE_TEST=OFF \
    -DSONATA_REPORT_ENABLE_MPI=ON \
    -DCMAKE_PREFIX_PATH="$CONDA_PREFIX" \
    -DHDF5_ROOT="$CONDA_PREFIX" \
    -DHDF5_PREFER_PARALLEL=ON \
    -DMPI_C_COMPILER="$CONDA_PREFIX/bin/mpicc" \
    -DMPI_CXX_COMPILER="$CONDA_PREFIX/bin/mpicxx"
```

During configuration, CMake should report that MPI and parallel HDF5 were found and that the *parallel reporting implementation* will be used. If the output instead says that the serial reporting implementation will be used, stop and check the HDF5 installation before compiling.

Then build and install:

```
cmake --build build -j"${nproc}"
cmake --install build
```

A useful optional check is:

```
ldd "$CONDA_PREFIX/lib/libsonatareport.so" | grep -E 'hdf5|mpi'
```

Both HDF5 and MPI should resolve to the intended environment or installation.

## 4.4 Build NEURON/CoreNEURON with reporting enabled

When compiling NEURON/CoreNEURON, include the normal CoreNEURON and MPI options together with reporting support:

```
-DNRN_ENABLE_CORENEURON=ON \
-DNRN_ENABLE_MPI=ON \
-DCORENRN_ENABLE_REPORTING=ON
```

A simplified CMake configuration is:

```
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX="$CONDA_PREFIX" \
  -DNRN_ENABLE_CORENEURON=ON \
  -DNRN_ENABLE_MPI=ON \
  -DCORENRN_ENABLE_REPORTING=ON
```

Additional compiler, Python, MPI, and platform-specific options can be added as required by the local installation.

For general GPU simulation support, CoreNEURON can additionally be compiled with:

```
-DCORENRN_ENABLE_GPU=ON
```

There is no separate “GPU HDF5” option for this LFP workflow. GPU execution and parallel HDF5 reporting are separate layers. At the time of writing, the tested NEURON/CoreNEURON 9.0.1 GPU LFP report remains zero-valued even when parallel HDF5 is configured correctly, so the validated LFP configuration in this repository remains `cfg.gpu=False`.

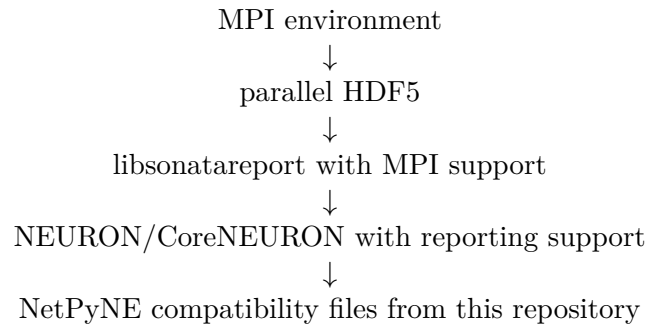
## 4.5 Runtime checks

For an MPI LFP run, check the reporting messages printed when CoreNEURON starts. The reporter should initialize its *parallel* implementation. If it initializes a serial implementation, the run should not be used for network LFP analysis until the HDF5/SONATA configuration is corrected.

If the shared libraries cannot be found at runtime, the environment library path can be extended with:

```
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:$LD_LIBRARY_PATH"
```

The recommended installation order is therefore:



## 5 Running and locating the LFP output

No special LFP command is required at runtime beyond the normal NetPyNE simulation workflow.

During `sim.setupRecording()`, the customized NetPyNE setup code calculates the extracellular transfer-resistance matrices and registers the corresponding LFP mappings with CoreNEURON.

During `sim.runSim()`, the customized run code provides CoreNEURON with the generated SONATA report configuration.

After the simulation finishes, CoreNEURON creates a temporary directory named:

`coreneuron_lfp/`

inside the simulation output directory. This folder contains the native CoreNEURON SONATA report, including:

```

netpyne_lfp.h5
report.conf
sim.conf

```

The file `netpyne_lfp.h5` contains the LFP contribution produced by CoreNEURON.

### 5.1 Minimal simulation configuration

A minimal CPU/CoreNEURON LFP configuration is:

```

cfg.coreneuron = True
cfg.gpu = False

cfg.recordLFP = [[0.0, 0.0, 5.0]]
cfg.recordLFPStep = 0.05 # ms

```

For strict sample-by-sample comparisons with the standard NetPyNE/NEURON path, it is convenient to use the same recording interval for both paths, for example `cfg.recordLFPStep = cfg.recordStep`.

### 5.2 Quick MPI validation

Before running long simulations, use a short test and verify the following:

1. `h5pcc -showconfig` reports `Parallel HDF5: ON`;
2. the `libsonatareport` CMake configuration selects the parallel reporting implementation;
3. CoreNEURON initializes the parallel reporter at runtime;
4. the generated HDF5 report contains the expected compartment-cell GIDs.

The last point can be checked quickly with:

```
import h5py

with h5py.File("netpyne_lfp.h5", "r") as f:
    node_ids = f["report/All/mapping/node_ids"][:]
    print("reported cells:", len(node_ids))
    print("first GIDs:", node_ids[:10])
```

### 5.3 Importing the LFP into the NetPyNE output

The CoreNEURON SONATA report is initially stored separately from the standard NetPyNE simulation data. To include the final LFP directly in the NetPyNE output file, the SONATA report can be loaded after `sim.gatherData()` and before `sim.saveData()`.

The recommended simulation sequence is therefore:

```
sim.runSim()

sim.gatherData()

# Load CoreNEURON LFP into NetPyNE data
if cfg.coreneuron and cfg.recordLFP:

    lfp, info = load_coreneuron_lfp(
        sim._coreLFP_report_file
    )

    lfp = np.asarray(lfp)

    sim.allSimData['LFP'] = lfp
    sim.simData['LFP'] = lfp

sim.saveData()

sim.analysis.plotData()
```

The variable:

```
sim._coreLFP_report_file
```

is created automatically by the customized `coreneuron_lfp.py` module and contains the complete path to the generated SONATA LFP report.

For example, if a batch simulation defines:

```
b.batchLabel = 'experiment_01'
b.saveFolder = '../data/' + b.batchLabel
```

the CoreNEURON LFP file is located automatically under:

```
../data/experiment_01/coreneuron_lfp/netpyne_lfp.h5
```

without requiring any change to the simulation script.

## 5.4 Reading the CoreNEURON SONATA report

The following helper function was used to read the CoreNEURON LFP report:

```
def load_coreneuron_lfp(filename):

    with h5py.File(filename, "r") as f:

        group = f["report/All"]

        data = group["data"][:]
        element_ids = group["mapping/element_ids"][:]
        node_ids = group["mapping/node_ids"][:]
        index_pointers = group["mapping/index_pointers"][:]
        time_info = group["mapping/time"][:]

    n_cells = len(node_ids)

    n_elements = data.shape[1]

    n_electrodes = n_elements // n_cells

    if n_cells * n_electrodes != n_elements:
        raise ValueError(
            "LFP mapping is not regular: "
            f"{n_elements} elements for {n_cells} cells"
        )

    # time x cell x electrode
    per_cell_lfp = data.reshape(
        data.shape[0],
        n_cells,
        n_electrodes,
    )

    # Sum contribution from every cell.
    #
    # Result:
    # time x electrode
    network_lfp = per_cell_lfp.sum(axis=1)

    return network_lfp, time_info
```

The required imports are:

```
import h5py
import numpy as np
```

CoreNEURON stores one LFP contribution for each cell and electrode combination. Therefore, the raw SONATA report contains data organized conceptually as:

```
time x cell x electrode
```

The helper function reshapes this information and sums the contribution of every cell, producing the final network LFP:

```
time x electrode
```

This final array is then assigned to:

```
sim.allSimData['LFP']
```

before NetPyNE saves the simulation.

As a result, the standard NetPyNE `.pk1` output contains the LFP together with the other simulation variables, and the SONATA HDF5 file does not need to be handled separately during later analysis.

## 5.5 Putting the LFP into the normal NetPyNE output

A convenient final workflow is to read the temporary SONATA report after `sim.gatherData()`, sum the per-cell contributions for each electrode, assign the result to `sim.allSimData['LFP']`, and then call `sim.saveData()`.

Conceptually:

```
sim.runSim()
sim.gatherData()

# Load/sum native CoreNEURON LFP on rank 0
# sim.allSimData['LFP'] = network_lfp

sim.saveData()
```

After the NetPyNE data file has been saved successfully, the temporary `coreneuron_lfp` directory may be deleted if the raw SONATA report is not needed.

## 6 How the workaround works internally

The remainder of this document describes the implementation in more detail. It is not required for normal use of the provided files.

### 6.1 Traditional NetPyNE LFP path

In the regular NEURON path, NetPyNE creates membrane-current pointers for every compartment. During the simulation, the LFP callback obtains the current vector for each cell and calculates

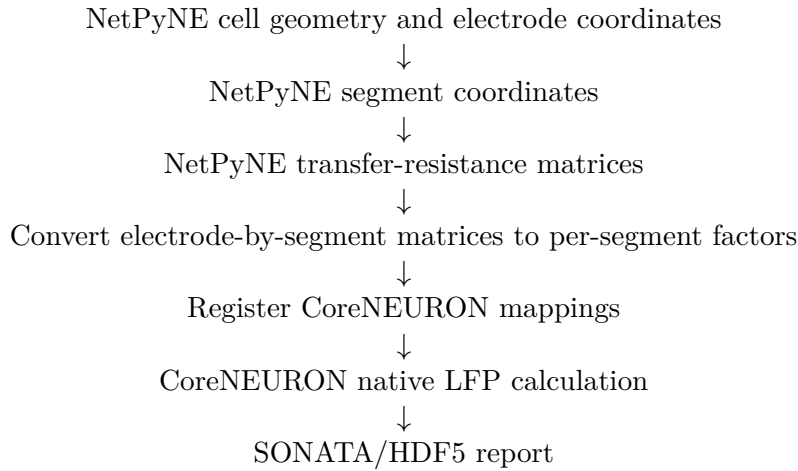
$$\mathbf{v}_{\text{cell}}(t) = \mathbf{R}_{\text{cell}} \mathbf{i}_{\text{mem,cell}}(t), \quad (2)$$

then sums these contributions into the total network LFP.

The important architectural feature is that this calculation is performed through NetPyNE/NEURON recording objects during the simulation. The CoreNEURON workaround intentionally avoids relying on that traditional runtime path.

### 6.2 New CoreNEURON path

The modified workflow is:



This preserves NetPyNE’s existing extracellular geometry calculations while moving only the runtime LFP evaluation and report writing into CoreNEURON.

## 7 Details of the `setup.py` modification

The customized `setupRecordLFP()` retains the common NetPyNE geometry setup:

- cell shapes are defined if necessary;
- segment coordinates are calculated;
- `RecXElectrode` is constructed from `cfg.recordLFP`;
- transfer resistance is calculated for every local compartment cell.

The implementation then verifies that each local GID has a transfer-resistance matrix before native LFP registration begins. This prevents a partially initialized report from being silently accepted.

For a CoreNEURON LFP run, `setup.py` calls `setup_core_lfp()` and then returns before the standard NetPyNE `PtrVector/imembVec` recording structures are created. That separation is intentional: the traditional NetPyNE LFP recorder is used for normal NEURON execution, while the native CoreNEURON path is used for CoreNEURON execution.

The regular NEURON behavior is therefore preserved when `cfg.coreneuron` is false.

## 8 Details of coreneuron\_lfp.py

### 8.1 Transfer-resistance conversion

For each cell, NetPyNE stores a transfer-resistance array with shape

$$N_{\text{electrodes}} \times N_{\text{segments}}. \quad (3)$$

The helper traverses the cell’s sections and segments in the same order used to build the NetPyNE transfer-resistance matrix. For each segment it records:

- the NetPyNE/NEURON section identifier;
- the NEURON node index returned by the segment;
- the corresponding column of the transfer-resistance matrix.

The code explicitly checks that the number of traversed segments equals the number of transfer-resistance columns. This is one of the most important correctness checks in the workaround.

The matrix is then reorganized from

```
electrode 0: segment0 segment1 segment2 ...
electrode 1: segment0 segment1 segment2 ...
electrode 2: segment0 segment1 segment2 ...
```

into the layout expected for registration:

```
segment 0: electrode0 electrode1 electrode2 ...
segment 1: electrode0 electrode1 electrode2 ...
segment 2: electrode0 electrode1 electrode2 ...
```

Those factors are registered with `ParallelContext.nrnbbcore_register_mapping()`.

### 8.2 Section classes

The helper groups section names into CoreNEURON section classes such as `soma`, `dend`, `apic`, `axon`, `myelin`, `ais`, and `node`. Unrecognized names fall back to the generic `cell` class.

This allows heterogeneous NetPyNE morphologies to be translated into the mapping categories expected by CoreNEURON.

### 8.3 Report configuration

After the mappings are registered, the helper gathers the compartment-cell GIDs across MPI ranks and writes a CoreNEURON report configuration. The report is defined as an LFP report on `i_membrane` using SONATA format.

The helper creates:

- `report.conf`: report metadata, time range, report interval, GID target list, buffering information, and spike-report metadata required by the parser;
- `sim.conf`: output path and pointer to `report.conf`.

The generated paths are attached to the NetPyNE `sim` object so the run code and user post-processing do not need to reconstruct them independently.

## 9 Details of the run.py modification

The customized `run.py` makes two important changes to the LFP workflow.

First, the traditional NetPyNE LFP event callback is scheduled only when CoreNEURON is not being used. This prevents the standard Python-side LFP calculation from being mixed with the native CoreNEURON path.

Second, immediately before enabling CoreNEURON execution, `runSim()` checks that the LFP setup produced a simulation configuration and assigns it to:

```
coreneuron.sim_config = sim._coreLFP_sim_config
```

CoreNEURON is then enabled normally and `psolve()` runs the simulation. The native reporting engine uses the mappings and report configuration prepared earlier.

## 10 Voltage-trace recording with CoreNEURON

This project also uses a small NetPyNE compatibility change for voltage trajectories. In the tested CoreNEURON workflow, a `Vector.record()` request with an explicit custom recording interval can fail to return the complete trajectory after the CoreNEURON run. The practical workaround is to omit the explicit recording interval when CoreNEURON is active and keep the original NetPyNE behavior for standard NEURON runs.

Conceptually, the recording code becomes:

```
if sim.cfg.coreneuron:
    trace_vector.record(ptr)
else:
    trace_vector.record(ptr, sim.cfg.recordStep)
```

For consistency, use:

```
cfg.recordStep = cfg.dt
```

when voltage traces are required with CoreNEURON. This records the CoreNEURON trajectory at the simulation time step; the saved trace can be downsampled afterward if a coarser analysis interval is desired. The standard NEURON path can continue to use the explicit `recordStep` argument.

This behavior is consistent with the CoreNEURON trajectory-recording limitation discussed by the NEURON developers in [NEURON discussion #2090](#).

## 11 Current limitations

### 11.1 GPU-native LFP

The most important current limitation is the CPU/GPU difference observed during validation. With the same model, transfer-resistance factors, electrodes, mappings, and report configuration:

- CoreNEURON CPU produced nonzero LFP;
- CoreNEURON GPU produced a correctly structured report whose LFP data were zero.

This indicates that the NetPyNE-side mapping itself is functional, while the tested GPU reporting path has a separate issue. Until that behavior is clarified or fixed upstream, the recommended LFP configuration is `cfg.gpu=False`.

## 11.2 Features intentionally not supported by this first implementation

The provided helper currently raises `NotImplementedError` for several NetPyNE options that depend on the older membrane-current gathering path:

- `saveIMembrane`;
- `saveLFPCells`;
- `saveLFP pops`.

The implemented path focuses on total native CoreNEURON LFP recording.

This is a project-level workaround tested with Python 3.10, NetPyNE 1.1.1, NEURON/CoreNEURON 9.0.1, Open MPI 5.0.10, and parallel HDF5 2.2.0. It is not an official NetPyNE or CoreNEURON patch. Behavior may change in newer software versions.